Anexo 26. Entorno de Ejecución, Dockerfiles, Variables y Volúmenes

Capítulo: 5 (Implementación y Evaluación PoC) | **Objetivo Específico:** OE4 | **Investigador:** Alvaro Felipe Lopez Russi | **Entidad:** ITY SAS / Universidad Católica de Colombia

1. Especificaciones de Hardware y Plataforma Host

La prueba de concepto del sistema multiagente se diseñó e implementó para ejecutarse de manera autónoma en una estación de trabajo representativa de una agencia de marketing digital, sin depender obligatoriamente de infraestructura en la nube pública.

Ficha Técnica del Entorno Host de Evaluación:

- **Procesador:** AMD Ryzen 7 5700X / Intel Core i7 (8 núcleos, 16 hilos a 3,8 GHz base).
- **Memoria RAM:** 32 GB DDR4 a 3200 MHz (asignación dinámica a Docker).
- **Aceleración Gráfica (GPU):** NVIDIA GeForce RTX 3060 (12 GB VRAM GDDR6, CUDA Compute 8.6) para inferencia local en Ollama.
- **Almacenamiento:** Unidad de Estado Sólido NVMe M.2 PCIe 4.0 de 1 TB (lectura 5000 MB/s).
- **Sistema Operativo:** Windows 11 Pro (64 bits, compilación 23H2) con subsistema Linux WSL2 (Ubuntu 22.04 LTS kernel 5.15).
- **Motor de Contenedores:** Docker Desktop 28.5.1 sobre backend WSL2 Linux Engine con Docker Compose v2.

---

2. Inventario de los 14 Microservicios Contenerizados

La arquitectura desacopla el sistema en 14 contenedores conectados a través de una red puente interna (`proyectomultiagente_default`):

#	Nombre del Servicio	Imagen Base / Dockerfile	Puerto Host:Container	Protocolo / Rol Principal	Dependencias (`depends_on`)
1	`postgres`	`pgvector/pgvector:pg16`	`5432:5432`	Base de datos relacional y vectorial.	Ninguna (Servicio Base)
2	`redis`	`redis:7-alpine`	`6379:6379`	Checkpoints LangGraph y caché semántica.	Ninguna (Servicio Base)

#	Nombre del Servicio	Imagen Base / Dockerfile	Puerto Host:Container	Protocolo / Rol Principal	Dependencias (depends_on)
3	`rabbitmq`	`rabbitmq:3.13-management-alpine`	`5672:5672`, `15672:15672`	Broker de mensajería AMQP y consola web.	Ninguna (Servicio Base)
4	`llm-gateway`	`Dockerfile.llm_gateway`	`8003:8003`	Pasarela REST de modelos de lenguaje.	`redis`
5	`orchestrator`	`Dockerfile.orchestrator`	`8002:8002`	Coordinación, guarda HITL y state machine.	`postgres`, `redis`, `rabbitmq`
6	`agente-ideacion`	`Dockerfile.agente_ideacion`	`8000:8000`	Worker AMQP y grafo LangGraph de ideación.	`orchestrator`, `llm-gateway`
7	`agente-publicacion`	`Dockerfile.agente_publicacion`	`8001:8001`	Worker AMQP y bifurcación multicanal.	`orchestrator`, `llm-gateway`
8	`tool-client`	`Dockerfile.tool_client`	`8010:8010`	Perfiles de clientes y datos de marca.	`postgres`
9	`tool-trends`	`Dockerfile.tool_trends`	`8011:8011`	Ingesta de tendencias de mercado (RSS).	`postgres`
10	`tool-competitors`	`Dockerfile.tool_competitors`	`8012:8012`	Monitoreo y benchmarking de marcas.	`postgres`
11	`tool-news`	`Dockerfile.tool_news`	`8013:8013`	Filtrado temático de noticias.	Ninguna
12	`tool-trending`	`Dockerfile.tool_trending`	`8014:8014`	Detección de hashtags y audios virales.	`tool-trends`
13	`tool-rag`	`Dockerfile.tool_rag`	`8015:8015`	Búsqueda por similitud semántica.	`postgres`, `llm-gateway`
14	`tool-postiz`	`Dockerfile.tool_postiz`	`8016:8016`	Adaptador REST para despacho multicanal.	Ninguna

---

3. Configuración de Variables de Entorno y Redes

El despliegue se parametriza de forma desacoplada mediante un archivo `.env` en la raíz del proyecto:

```
# Entorno General
ENVIRONMENT=production
LOG_LEVEL=INFO

# Persistencia Relacional y Vectorial
POSTGRES_HOST=postgres
POSTGRES_PORT=5432
POSTGRES_DB=multiagente_db
POSTGRES_USER=postgres
POSTGRES_PASSWORD=postgres_secure_pass

# Broker de Mensajería
RABBITMQ_HOST=rabbitmq
RABBITMQ_PORT=5672
RABBITMQ_USER=guest
RABBITMQ_PASS=guest

# Memoria de Alta Velocidad y Cache
REDIS_HOST=redis
REDIS_PORT=6379

# Enrutamiento de Modelos
LLM_GATEWAY_URL=http://llm-gateway:8003
OLLAMA_BASE_URL=http://host.docker.internal:11434
PRIMARY_MODEL=gemma3:4b
FALLBACK_MODEL=mistral-nemo:latest
EMBEDDING_MODEL=nomic-embed-text:latest
```

Persistencia de Volúmenes:

Se verificó que los datos no se destruyan ante reinicios en caliente mediante volúmenes declarados en `docker-compose.yml`: * `postgres_data:/var/lib/postgresql/data` (Garantiza inmutabilidad de marcas, briefs y publicaciones). * `rabbitmq_data:/var/lib/rabbitmq` (Garantiza durabilidad de colas y mensajes pendientes). * `redis_data:/data` (Mantiene puntos de control de LangGraph).